5-2 Milestone 4 Journal

Jamie Kutzer

Southern New Hampshire University

CS499: Computer Science Capstone

October 4, 2026

5-2 Milestone 4 Journal

The artifact I selected for my database enhancement is the Travlr Getaways application, which I originally created in CS 465. Travlr is a full-stack web application that allows users to view travel packages and provides an administrative interface for managing trip information. The application uses Angular for the administrative interface, Node.js and Express for the API, and MongoDB with Mongoose for storing and managing trip data. I selected Travlr for this enhancement because its existing database functionality provided an opportunity to improve how data is validated, stored, and protected while continuing to build on the same artifact used for my previous enhancements.

For this enhancement, I strengthened the Mongoose schema and database operations used to manage trip data. The length and perPerson fields were changed from strings to numbers because they represent numeric values, and minimum values were added to prevent invalid values from being stored. I also added trimming to several text fields to improve consistency in stored data. The trip code field was changed to use a unique index so duplicate trip identifiers cannot be stored. In addition, I updated the trip update operation to run Mongoose validators so the same database rules are enforced when an existing trip is modified. Error handling was also improved so validation errors, duplicate data, missing trips, and unexpected server errors return more appropriate HTTP responses.

This enhancement primarily demonstrates progress toward Course Outcomes Four and Five. Outcome Four is demonstrated through the use of MongoDB, Mongoose schemas, database validation, indexing, and API practices to improve the reliability and usefulness of the application. Outcome Five is demonstrated by adding another layer of protection for application data. The Angular interface already performs validation, but client-side validation should not be

the only protection for stored information. The enhanced Mongoose model independently validates data even when a request is sent directly to the API. This helps prevent invalid or duplicate information from being stored and demonstrates a stronger data-integrity and security mindset. These enhancements support my original outcome-coverage plan, so I do not currently need to make changes to that plan.

One of the most useful parts of this enhancement was seeing the difference between validation at the user interface and validation at the database level. During testing, the Angular application correctly prevented an invalid trip length from being submitted. I then used Postman to send a request directly to the API with a trip length of zero. Mongoose rejected the request based on the schema validation rules, and the existing valid value remained unchanged in the database. This demonstrated why validation at multiple layers is important rather than assuming all data will come through the expected user interface.

I also encountered a challenge while implementing the unique trip code. Although I changed the Mongoose schema to specify that the trip code should be unique, testing showed that a duplicate code could still be stored. I investigated the indexes in MongoDB Compass and discovered that the existing code_1 index had been created previously as a regular, non-unique index. After removing the duplicate test data and the old index, restarting the application allowed Mongoose to recreate the index with the unique constraint. A second test then correctly rejected the duplicate trip code. This process helped me better understand the relationship between a Mongoose schema and the indexes that already exist in MongoDB. Overall, the enhancement improved the integrity of Travlr's stored data while giving me more experience with database validation, indexing, API error handling, and testing database behavior independently from the application's user interface.